ENVIRONMENTAL VARIABLES AND CRONJOBS

Environment Variables & Cron Jobs in Linux

A Step-by-Step Implementation Guide

Table of Contents

1. Understanding Environment Variables
2. Viewing and Setting Variables in a Shell
3. Making Variables Permanent
4. Introducing Cron
5. Critical: Cron's Minimal Environment
6. Example 1: Backup Script (Failure & Fix)
7. Example 2: Python Virtual Environment in Cron
8. Example 3: Setting Variables Inside Crontab
9. Example 4: Logging and Output Redirection

10. Example 5: Debugging with env -i

11. Best Practices Summary

12. Robust Crontab Template

---

1. What Is an Environment Variable?

An environment variable is a named value inherited by child processes. It acts as configuration metadata for the shell and applications.

Common Variables:

Variable Typical Value Purpose

HOME /home/alice User home directory

USER alice Current username

SHELL /bin/bash Login shell interpreter

PATH /usr/local/bin:/usr/bin:/bin Search path for executables

LANG en_US.UTF-8 System language/locale

---

2. Viewing and Setting Variables

Open a terminal to test the following commands:

```bash
# Print a specific variable
echo "My home directory is: $HOME"

# List all environment variables currently set
env

# Set a variable (local to this shell session)
MY_NAME="Alice"
echo "Hello, $MY_NAME"

# Export the variable (makes it available to child processes/scripts)
export MY_NAME
```

---

3. Making Variables Permanent

· User-Specific: Add the export command to ~/.bashrc or ~/.profile.

```
bash
echo 'export EDITOR=nano' >> ~/.bashrc
source ~/.bashrc
```

· System-Wide: Add VAR=value (no export) to /etc/environment.

---

4. Cron Syntax Overview

Cron runs scheduled commands. Edit your user’s crontab using:

bash

crontab -e

Syntax Format:

```
*  *  *  *  *  command_to_execute
-  -  -  -  -
|  |  |  |  |
|  |  |  |  +----- Day of week (0-7) (Sunday=0)
|  |  |  +-------- Month (1-12)
|  |  +----------- Day of month (1-31)
|  +-------------- Hour (0-23)
+----------------- Minute (0-59)
```

---

5. The Critical Difference: Cron's Environment

Cron does NOT read .bashrc, .profile, or .bash_profile.

It runs jobs with a bare minimum environment (often just HOME, LOGNAME, and a limited PATH like /usr/bin:/bin). This is the #1 reason cron jobs fail while working perfectly in the terminal.

Simulate Cron's Environment for Testing:

```
bash
env -i HOME=$HOME /bin/sh -c 'echo $PATH'
```

---

6. Example 1: Backup Script (Real-World Fix)

Problematic Script (~/backup.sh):

bash

```
#!/bin/bash
# This fails in cron because 'date' and 'mkdir' might not be found
DEST="$HOME/backups"
mkdir -p "$DEST"
cp -r "$HOME/Documents" "$DEST/Documents-$(date +%Y%m%d)"
```

Cron Entry (Will Likely Fail):

cron

```
0 2 * * * /home/alice/backup.sh
```

Solution A: Use Absolute Paths (Robust)

bash

```
#!/bin/bash
DEST="/home/alice/backups"
```

```
/bin/mkdir -p "$DEST"
/bin/cp -r "/home/alice/Documents" "$DEST/Documents-$(/bin/date +%Y%m%d)"
```

Solution B: Define PATH at Top of Crontab

```
cron
PATH=/usr/local/bin:/usr/bin:/bin
0 2 * * * /home/alice/backup.sh
```

---

7. Example 2: Python Virtual Environment

Wrong Approach:

```
cron
0 8 * * * python /home/alice/scripts/report.py
```

Correct Approach: Use Full Path to Venv Python

cron

0 8 * * * /home/alice/venv/bin/python /home/alice/scripts/report.py

---

8. Example 3: Setting Environment Variables in Crontab

You can define variables directly in the crontab file. This is useful for API keys or specific configs.

cron

 # Global Crontab Settings

MAILTO=admin@example.com

PATH=/usr/local/bin:/usr/bin:/bin

MY_API_KEY="abc123xyz"

 # Daily report

0 7 * * * /home/alice/scripts/daily_report.sh

---

9. Example 4: Output Redirection & Logging

By default, cron emails the output of the job. To store it in a file instead:

cron

 # Append stdout and stderr to a log file

0 3 * * * /home/alice/backup.sh >> /home/alice/logs/backup.log 2>&1

---

10. Example 5: Debugging with env -i

Before trusting a script to cron, test it in a cron-like void:

bash

env -i HOME=$HOME /bin/sh -c 'cd /home/alice && /home/alice/backup.sh'

If this command works, cron will work. If it fails, you will see exactly why (e.g., "command not found").

---

11. Best Practices Summary

Practice Reason

Use Absolute Paths /usr/bin/python instead of python

Set PATH in Crontab Prevents "command not found" errors

Wrap in Shell Script Easier to test and version control

Never rely on ~ Use /home/username explicitly

Log Output >> /var/log/mycron.log 2>&1

Set MAILTO Receive error alerts via system mail

---

12. Final Template: A Production-Ready Crontab

```cron
# Environment for all jobs
SHELL=/bin/bash
PATH=/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin
MAILTO=alerts@example.com

# 1. Database Backup (1:00 AM Daily)
0 1 * * * /home/alice/scripts/db_backup.sh >> /home/alice/logs/db.log 2>&1

# 2. Python Weekly Report (Sunday 8:00 AM)
0 8 * * 0 /home/alice/venv/bin/python /home/alice/scripts/weekly_report.py
```

 # 3. Log Rotation (Monday 2:00 AM)

0 2 * * 1 /usr/bin/find /home/alice/logs -type f -name "*.log" -mtime +30 -delete